# Dossier de conception – Plateforme de gestion scolaire

Oct 20, 2026 · @Amadou SENE

## 1. Vision et périmètre

La plateforme gère de bout en bout la vie d'un établissement, du CI à la Terminale, en mode SaaS multi-écoles : une seule installation sert plusieurs écoles dont les données restent strictement isolées. Ce choix permet de commencer avec une école pilote, puis d'ouvrir à d'autres établissements ou à un groupe scolaire sans réécrire le logiciel.

**Niveaux couverts** (système sénégalais, paramétrable pour d'autres pays) :

- Primaire : CI, CP, CE1, CE2, CM1, CM2 — bulletins par compétences ou par matière, examen CFEE
- Moyen (collège) : 6e à 3e — notes par matière et coefficients, examen BFEM
- Secondaire (lycée) : 2nde, 1re, Tle avec séries (L, S, etc.) — coefficients par série, Baccalauréat

**Acteurs et ce qu'ils font dans l'outil**

| Acteur | Usage principal | Canal |
|---|---|---|
| Super-administrateur plateforme | Crée les écoles, gère les abonnements | Web |
| Directeur / proviseur | Pilote, valide bulletins, consulte tableaux de bord | Web |
| Secrétariat / scolarité | Inscriptions, dossiers élèves, certificats | Web |
| Censeur / surveillant général | Absences, retards, billets d'entrée, discipline | Web + tablette |
| Enseignant | Appel, notes, cahier de textes, devoirs | Web + mobile |
| Comptable / intendant | Frais de scolarité, paiements, relances | Web |
| Parent | Suivi de ses enfants, paiements, messages | Mobile (PWA) |
| Élève (collège/lycée) | Emploi du temps, notes, devoirs | Mobile (PWA) |

**Objectifs mesurables de la version 1**

- Un parent voit une absence ou un retard de son enfant en moins de 5 minutes après sa saisie.
- Un bulletin de trimestre est généré pour toute une classe en moins de 2 minutes.
- L'application reste utilisable avec une connexion 3G lente (pages < 500 Ko, mode hors-ligne pour l'appel).
- Disponibilité cible : 99,5 % pendant l'année scolaire.

## 2. Modules fonctionnels

Le logiciel est découpé en 12 modules métier indépendants ; chacun correspond à un module de code isolé (section 3). Priorité : P1 = version 1, P2 = version 2, P3 = plus tard.

| Module | Fonctions clés | Priorité |
|---|---|---|
| Établissement & année scolaire | Paramètres de l'école, cycles, niveaux, séries, trimestres/semestres, calendrier, jours fériés | P1 |
| Inscriptions & dossiers élèves | Pré-inscription en ligne, réinscription, matricule, pièces jointes, fratries, transferts, certificats de scolarité | P1 |
| Classes & affectations | Classes, effectifs, affectation élèves, professeur principal, passage en classe supérieure | P1 |
| Personnel | Enseignants, administratifs, contrats, matières enseignées, volume horaire | P1 |
| Emplois du temps | Grille hebdomadaire par classe/enseignant/salle, détection de conflits, remplacements | P2 |
| Vie scolaire & assiduité | Appel par séance, absences, retards, justificatifs, **billets d'entrée en classe**, sanctions, exclusions | P1 |
| Évaluations & notes | Devoirs, compositions, coefficients par niveau/série, moyennes, rangs, appréciations, conseils de classe | P1 |
| Bulletins & documents | Bulletins PDF trimestriels/semestriels, relevés, attestations, signature numérique, QR de vérification | P1 |

| Module | Fonctions clés | Priorité |
|---|---|---|
| Finances & scolarité | Grilles de frais (inscription, mensualités, cantine, transport), échéanciers, réductions, reçus, relances, mobile money | P1 |
| Espace parent & élève | Suivi multi-enfants, notes, absences, billets, devoirs, paiements, messagerie, notifications | P1 |
| Communication | Annonces, SMS/WhatsApp/e-mail ciblés, convocations, sondages | P2 |
| Pilotage & statistiques | Tableaux de bord : effectifs, taux de réussite, assiduité, recouvrement ; export pour l'inspection | P2 |

Modules P3 envisagés : cantine, transport scolaire, bibliothèque, infirmerie, examens blancs, paie du personnel.

## Billets d'entrée en classe

Un élève en retard ou revenant d'absence ne rentre en classe qu'avec un billet numérique émis par la vie scolaire ; l'enseignant le valide, le parent est prévenu. Le circuit :

1. L'élève se présente au bureau du surveillant (retard, retour d'absence, retour d'exclusion, sortie infirmerie).
2. Le surveillant recherche l'élève (nom, matricule ou scan de sa carte QR) ; l'historique récent s'affiche (retards du mois, absences non justifiées).
3. Il choisit le motif, indique si le justificatif est fourni, et émet le billet : numéro unique, heure, séance concernée, QR code.
4. Le billet est imprimé sur ticket thermique ou affiché sur le téléphone ; l'absence de la séance en cours est convertie en « retard autorisé ».
5. L'enseignant scanne ou saisit le billet dans son appel : l'élève passe « présent (entré à HH:MM) ».
6. Le parent reçoit une notification (application, SMS en secours) avec le motif et l'heure.

Règles paramétrables par école :

- Seuil de retards (ex. 3 par mois) déclenchant une convocation automatique des parents.
- Billet refusé sans justificatif après N jours d'absence ; renvoi vers le censeur.
- Un billet ne vaut que pour la journée et la séance indiquées (expiration automatique).
- Billet non scanné sous 15 minutes : alerte au surveillant (élève pas arrivé en classe).

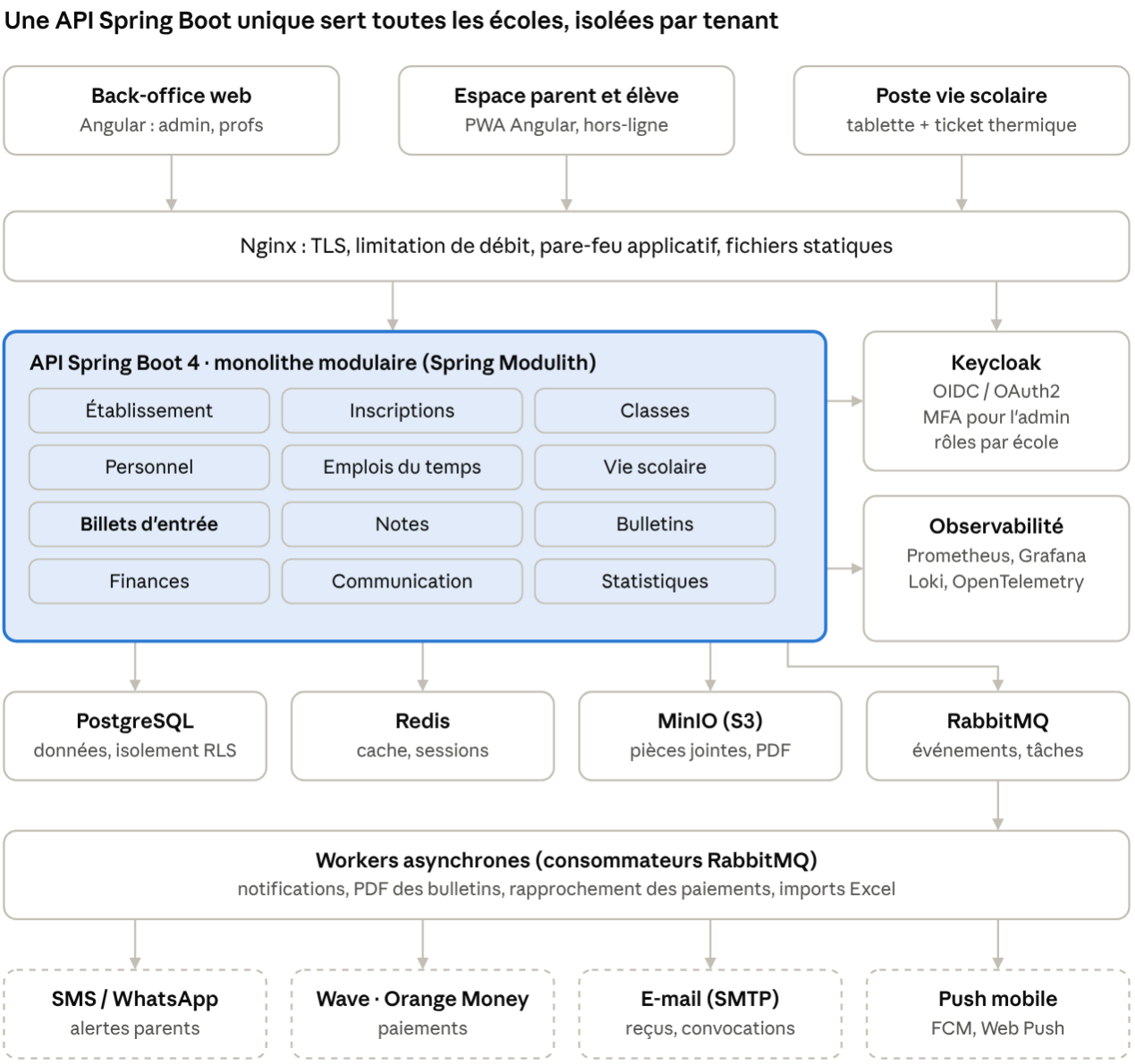
### Espace parent

Un compte parent unique donne accès à tous ses enfants, même inscrits dans plusieurs écoles de la plateforme.

- Fil d'actualité par enfant : absences, retards, billets, notes, sanctions, devoirs.
- Bulletins téléchargeables, évolution des moyennes.
- Situation financière : échéances, reçus, paiement Wave / Orange Money / carte.
- Justification d'absence en ligne (photo du certificat) et messagerie avec l'enseignant ou la vie scolaire.
- Prise de rendez-vous, lecture et accusé de réception des convocations.
- Interface en français, prévue pour l'ajout du wolof et de l'anglais ; option alertes SMS pour les parents sans smartphone.

## 3. Architecture technique

Recommandation : un **monolithe modulaire** Spring Boot + Angular, multi-tenant, déployé en conteneurs. Une seule application à déployer, mais découpée en modules aux frontières strictes ; les modules les plus sollicités (notifications, paiements) pourront devenir des services séparés sans réécriture. Les microservices dès le départ multiplieraient le coût d'exploitation sans bénéfice à l'échelle de quelques dizaines d'écoles.




architecture cible · 3 clients, 1 API modulaire, 4 services de données

Les clients passent tous par Nginx ; l'API vérifie chaque jeton auprès de Keycloak, écrit dans PostgreSQL et délègue les tâches lentes (SMS, PDF, paiements) aux workers via RabbitMQ.

## Choix techniques

| Couche | Choix | Pourquoi |
|---|---|---|
| Langage backend | Java 25 (LTS) | Support long, threads virtuels pour absorber les pics (publication des bulletins) |

| Couche | Choix | Pourquoi |
|---|---|---|
| Framework backend | Spring Boot 4.x, Spring Modulith, Spring Security, Spring Data JPA, Flyway, MapStruct | Standard industriel ; Modulith vérifie par des tests que les modules ne se mélangent pas |
| Frontend | Angular 22, Signals, Angular Material ou PrimeNG, @angular/pwa | Structure imposée, injection de dépendances, formulaires typés : adapté à une grosse application de gestion maintenue des années |
| Base de données | PostgreSQL 17 | Row-Level Security pour l'isolement des écoles, JSONB, recherche plein texte |
| Identité | Keycloak (OIDC) | MFA, gestion des sessions, fédération possible ; pas de mots de passe stockés dans l'API |
| Asynchrone | RabbitMQ + pattern outbox | Aucun SMS ni paiement perdu si un service externe est en panne |
| Fichiers | MinIO (compatible S3) | Justificatifs, photos, bulletins PDF ; bascule vers un cloud S3 sans changer le code |
| PDF | JasperReports ou Thymeleaf + OpenPDF | Modèles de bulletins personnalisables par école |

**Angular ou React ?** Angular est recommandé ici : son cadre imposé limite les dérives quand plusieurs développeurs travaillent des années sur des centaines de formulaires. React reste un bon choix si votre équipe le maîtrise déjà ; l'architecture backend ne change pas.

## Multi-tenant : une école = un tenant

- **Stratégie** : base et schéma partagés, chaque table métier porte une colonne `tenant_id`.
- **Double barrière** : Hibernate filtre automatiquement par tenant ( `@TenantId` ) et PostgreSQL applique une politique RLS ; une requête oubliée par un développeur ne peut pas lire une autre école.
- **Résolution du tenant** : sous-domaine ( `lycee-x.monapp.sn` ) vérifié contre la revendication `tenant` du jeton JWT ; incohérence = requête rejetée.
- **Grands clients** : possibilité de base dédiée pour un groupe scolaire qui l'exige, sans changer le code.

### Fonctionnement hors-ligne

L'appel et l'émission de billets fonctionnent sans connexion : les saisies sont stockées dans IndexedDB puis synchronisées. Chaque saisie porte un identifiant généré côté client, ce qui rend la synchronisation idempotente (aucun doublon si l'envoi est répété).

## 4. Modèle de données principal

Le cœur du modèle est l'**inscription** : un élève est rattaché à une classe pour une année scolaire donnée, et tout le reste (notes, absences, billets, frais) pointe vers cette inscription. On conserve ainsi l'historique complet d'un élève du CI à la Terminale sans jamais écraser une année passée.

| Domaine | Entités | Relations clés |
|---|---|---|
| Structure | `Tenant`, `Etablissement`, `AnneeScolaire`, `Periode`, `Cycle`, `Niveau`, `Serie`, `Classe`, `Salle` | Une classe = un niveau (+ série au lycée) pour une année |
| Personnes | `Personne`, `Eleve`, `Parent`, `Personnel`, `LienParental` | Un parent lié à N élèves, avec type (père, mère, tuteur) et droits (payeur, contact d'urgence) |
| Scolarité | `Inscription`, `DossierPiece`, `Transfert` | Élève × Classe × Année, statut (pré-inscrit, inscrit, transféré, radié) |
| Pédagogie | `Matiere`, `Coefficient`, `Enseignement`, `Seance`, `CahierTexte`, `Devoir` | Coefficient par niveau/série ; enseignement = prof × matière × classe |
| Évaluations | `Evaluation`, `Note`, `MoyennePeriode`, `Appreciation`, `Bulletin` | Note = élève × évaluation ; moyennes calculées puis figées à la validation |
| Vie scolaire | `Appel`, `Presence`, `Absence`, `Justificatif`, `BilletEntree`, `Sanction`, `Incident` | Billet lié à l'absence ou au retard qu'il régularise et à la séance d'entrée |
| Finances | `GrilleFrais`, `Echeance`, `Remise`, `Facture`, `Paiement`, `Recu`, `TransactionMobileMoney` | Échéances générées depuis la grille à l'inscription ; un paiement peut solder plusieurs échéances |

| Domaine | Entités | Relations clés |
|---|---|---|
| Communication | `Message`, `Notification`, `Annonce`, `Convocation` | Notification = destinataire × canal × statut d'envoi |
| Transverse | `Utilisateur`, `Role`, `JournalAudit`, `OutboxEvent` | Utilisateur lié à Keycloak par identifiant externe |

## Exemple : la table des billets d'entrée

```
CREATE TABLE billet_entree (
  id               UUID PRIMARY KEY,
  tenant_id        UUID        NOT NULL,
  numero           VARCHAR(20) NOT NULL,          -- ex. BE-2026-004512
  inscription_id   UUID        NOT NULL REFERENCES inscription(id),
  motif            VARCHAR(30) NOT NULL,          -- RETARD, RETOUR_ABSENCE,
RETOUR_EXCLUSION, INFIRMERIE
  absence_id       UUID        REFERENCES absence(id),
  seance_id        UUID        NOT NULL REFERENCES seance(id),
  justificatif_fourni BOOLEAN  NOT NULL DEFAULT FALSE,
  emis_par         UUID        NOT NULL REFERENCES utilisateur(id),
  emis_le          TIMESTAMPTZ NOT NULL,
  expire_le        TIMESTAMPTZ NOT NULL,
  statut           VARCHAR(15) NOT NULL,          -- EMIS, UTILISE, EXPIRE,
ANNULE
  utilise_le       TIMESTAMPTZ,
  valide_par       UUID        REFERENCES utilisateur(id),
  jeton_qr         VARCHAR(64) NOT NULL,          -- valeur signée HMAC, non
devinable
  version          INT         NOT NULL DEFAULT 0, -- verrou optimiste
  UNIQUE (tenant_id, numero)
);
ALTER TABLE billet_entree ENABLE ROW LEVEL SECURITY;
CREATE POLICY isolement_ecole ON billet_entree
  USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

### Règles générales du modèle

- Clés primaires UUID v7 (triables dans le temps, générables hors-ligne).
- Colonnes d'audit sur chaque table : `cree_le`, `cree_par`, `modifie_le`, `modifie_par`, et `version` pour le verrouillage optimiste.
- Suppression logique ( `supprime_le` ) pour les données scolaires ; jamais de suppression physique d'une note ou d'un paiement.

- Les montants sont stockés en entier (francs CFA, pas de décimales) ; devise en colonne pour d'autres pays.
- Les moyennes et rangs validés en conseil de classe sont figés : une note modifiée après validation exige une réouverture tracée.
- Index systématique commençant par `tenant_id` sur toutes les tables métier.

## 5. Sécurité et conformité

La plateforme manipule des données de mineurs et des paiements : la sécurité est conçue dès la première ligne de code, pas ajoutée à la fin. Le principe directeur est le **moindre privilège** : chaque utilisateur ne voit que son école, ses classes, ses enfants.

### Contrôle d'accès (RBAC + périmètre)

Un rôle dit *ce qu'on peut faire*, un périmètre dit *sur quelles données*. Les deux sont vérifiés côté serveur, jamais seulement dans l'interface.

| Rôle | Peut faire | Périmètre |
|---|---|---|
| Directeur | Tout lire, valider bulletins, gérer le personnel | Son établissement |
| Surveillant | Émettre/annuler des billets, saisir absences et sanctions | Les classes de son cycle |
| Enseignant | Faire l'appel, saisir des notes, valider des billets | Ses classes et ses matières uniquement |
| Comptable | Encaisser, émettre reçus, relancer | Finances de son établissement |
| Parent | Consulter, payer, justifier, écrire | Ses enfants liés uniquement |
| Élève | Consulter notes, emploi du temps, devoirs | Lui-même |

Côté code : `@PreAuthorize` sur chaque service + un `PermissionEvaluator` qui vérifie le périmètre (ex. l'enseignant enseigne-t-il dans cette classe ?). Les rôles sont stockés par école : une même personne peut être enseignante dans un lycée et parent dans une école primaire.

### Authentification

- Keycloak avec OIDC + PKCE pour les applications web ; jetons d'accès courts (5 min), jetons de rafraîchissement rotatifs.

- MFA obligatoire pour direction, comptabilité et super-administrateurs.
- Parents : connexion par numéro de téléphone + code OTP par SMS, ou mot de passe ; activation du compte par code remis par l'école (évite qu'un tiers se déclare parent).
- Verrouillage progressif après échecs, détection de connexions inhabituelles.

## Protection des données et de l'application

- TLS 1.2+ partout ; chiffrement au repos des volumes et sauvegardes ; champs sensibles (santé, situation familiale) chiffrés en colonne.
- Isolement des écoles par Hibernate + RLS PostgreSQL (section 3), testé automatiquement à chaque build.
- QR des billets et des bulletins signés (HMAC) : impossible de fabriquer un faux billet ou de falsifier un bulletin vérifiable.
- Protection OWASP Top 10 : requêtes paramétrées, validation des entrées, en-têtes de sécurité (CSP, HSTS), protection CSRF, limitation de débit sur la connexion et l'OTP.
- Fichiers déposés : types autorisés, taille maximale, analyse antivirus (ClamAV), accès par URL signée à durée limitée.
- Webhooks mobile money : vérification de signature, idempotence, rapprochement avant de marquer une échéance payée.
- Journal d'audit immuable : qui a modifié une note, annulé un billet, encaissé un paiement, et quand.
- Analyse de dépendances (OWASP Dependency-Check / Trivy) et test d'intrusion avant chaque mise en production majeure.

## Conformité au Sénégal

Le traitement relève de la loi n° 2008-12 du 25 janvier 2008 sur les données personnelles, contrôlée par la Commission de Protection des Données Personnelles (CDP) ; une révision du texte est évoquée, à suivre.

- Déclaration du traitement auprès de la CDP avant mise en service ; contrat de sous-traitance entre l'éditeur et chaque école.
- Consentement et information des parents à l'inscription ; droits d'accès, de rectification et d'opposition exerçables depuis l'espace parent.
- Durées de conservation définies (ex. absences : année en cours + 1 an ; notes et bulletins : archivage long) avec purge automatique.
- Hébergement : privilégier un centre de données au Sénégal ou en Afrique de l'Ouest ; tout transfert hors du pays à valider avec la CDP.

# 6. API, intégrations et notifications

L'API est REST/JSON, versionnée dans l'URL ( `/api/v1/...` ) et documentée automatiquement en OpenAPI ; le front Angular est généré depuis ce contrat, ce qui évite toute désynchronisation entre équipes.

## Conventions

- Ressources au pluriel, imbriquées au plus sur deux niveaux : `/api/v1/classes/{id}/eleves` .
- Pagination par curseur pour les grandes listes, filtres en paramètres ( `?annee=2026-2027&statut=INSCRIT` ).
- Erreurs au format RFC 9457 ( `application/problem+json` ) avec code métier lisible ( `BILLET_EXPIRE` ).
- En-tête `Idempotency-Key` obligatoire sur les créations sensibles (paiements, billets, appel hors-ligne).
- Verrouillage optimiste via `ETag` / `If-Match` : deux surveillants ne peuvent pas modifier le même billet en même temps.

## Exemples d'endpoints du module billets

| Méthode | Endpoint | Rôle | Effet |
|---|---|---|---|
| POST | `/api/v1/billets` | Surveillant | Émet un billet, notifie le parent |
| GET | `/api/v1/billets?date=2026-10-20&classe={id}` | Surveillant, enseignant | Billets du jour d'une classe |
| POST | `/api/v1/billets/scan` | Enseignant | Valide le QR, passe l'élève présent |
| POST | `/api/v1/billets/{id}/annulation` | Surveillant, censeur | Annule avec motif obligatoire (audit) |
| GET | `/api/v1/parents/me/enfants/{id}/billets` | Parent | Historique des billets de son enfant |

### Intégrations externes

Chaque fournisseur est caché derrière une interface (`SmsGateway`, `PaymentProvider`) : changer d'opérateur ne touche pas le code métier.

| Besoin | Fournisseurs envisagés | Remarque |
|---|---|---|
| Paiement mobile | Wave, Orange Money, Free Money (via agrégateur type PayDunya ou CinetPay) | Un agrégateur réduit le nombre de contrats et d'API à maintenir |
| SMS | Orange SMS API ou agrégateur local | Canal de secours pour parents sans smartphone |
| WhatsApp | WhatsApp Business Platform | Taux de lecture élevé ; modèles de messages à faire valider |
| E-mail | SMTP transactionnel | Reçus, convocations, bulletins |
| Push | Firebase Cloud Messaging, Web Push | Gratuit, prioritaire avant le SMS |
| Import | Excel/CSV | Reprise des listes d'élèves et notes existantes |

### Notifications : du moins cher au plus sûr

Un événement métier (billet émis, absence saisie, note publiée) est enregistré dans la table outbox dans la même transaction, puis publié sur RabbitMQ. Le worker de notification envoie d'abord une notification push ; sans accusé de lecture sous 10 minutes pour une alerte importante (absence, billet), il bascule sur SMS. Chaque parent choisit ses canaux et peut regrouper les notes en résumé quotidien. Les coûts SMS sont suivis par école et refacturés si besoin.

## 7. Scalabilité, déploiement et exploitation

L'application est sans état (stateless) : on ajoute des instances de l'API derrière Nginx quand la charge monte, sans rien modifier au code. Le dimensionnement initial vise 50 écoles et 30 000 élèves ; il passe à plusieurs centaines d'écoles en ajoutant des instances et un réplica de lecture.

## Pics de charge prévus et réponses

| Pic | Moment | Réponse technique |
|---|---|---|
| Appel du matin | 8 h – 8 h 30, toutes les classes | Saisie groupée en un appel API par classe, hors-ligne possible |
| Publication des bulletins | Fin de trimestre, tous les parents se connectent | PDF pré-générés par les workers, servis depuis MinIO + cache CDN |
| Rentrée / inscriptions | Septembre – octobre | File d'attente pour les imports, mise à l'échelle temporaire |
| Échéances de paiement | Début de mois | Webhooks traités en asynchrone, idempotents |

## Performance

- Cache Redis pour les référentiels peu changeants (classes, matières, emplois du temps).
- Requêtes de lecture lourdes (statistiques) sur un réplica PostgreSQL ou des vues matérialisées rafraîchies la nuit.
- Pool de connexions HikariCP + PgBouncer ; détection des requêtes N+1 en revue de code et en test.
- Front : chargement différé par module, images compressées, budget de 500 Ko au premier chargement de l'espace parent.

## Déploiement

1. **Démarrage (1 à 20 écoles)** : Docker Compose sur 2 serveurs (application + base), sauvegardes externes. Coût maîtrisé, exploitation simple.
2. **Croissance (20 à 200 écoles)** : Kubernetes managé (ou k3s), PostgreSQL managé avec réplica, mise à l'échelle automatique de l'API et des workers.
3. **Grande échelle** : extraction des modules notifications et paiements en services dédiés ; bases dédiées pour les plus gros clients.

Environnements : développement, recette (données anonymisées), pré-production, production.

### Intégration et livraison continues

- GitHub Actions ou GitLab CI : compilation, tests unitaires et d'intégration (Testcontainers avec un vrai PostgreSQL), tests d'architecture Modulith, analyse SonarQube, scan de sécurité.
- Images Docker signées, migrations Flyway appliquées automatiquement au déploiement.
- Déploiement progressif (rolling) sans interruption ; retour arrière en une commande.
- Fonctionnalités activables par école (feature flags) pour piloter une nouveauté sur une école avant de l'ouvrir à toutes.

### Observabilité et continuité

- Métriques (Micrometer → Prometheus → Grafana), journaux centralisés (Loki), traces (OpenTelemetry) ; chaque journal porte le `tenant_id` et un identifiant de requête.
- Alertes : taux d'erreur, latence p95 > 800 ms, file RabbitMQ qui grossit, échec d'envoi SMS ou de paiement.
- Sauvegardes PostgreSQL continues (WAL) avec restauration à un instant donné ; objectif de perte de données ⩽ 15 min, reprise ⩽ 4 h ; test de restauration chaque mois.

## 8. Structure du code et conventions

Chaque module métier suit la même organisation interne (architecture hexagonale légère) : le métier ne dépend ni de la base ni du web, ce qui le rend testable et durable. Un module n'accède à un autre que par son API publique ou par événement.

## Backend (Spring Boot, un seul dépôt)

```
backend/
├── src/main/java/sn/ecole/
|   ├── EcoleApplication.java
|   ├── shared/                 # tenant, sécurité, audit, erreurs, outbox
|   ├── etablissement/
|   ├── inscription/
|   ├── viescolaire/
|   |   ├── api/                # contrôleurs REST, DTO, mappers
|   |   ├── application/        # cas d'usage : EmettreBillet, ValiderBillet
|   |   ├── domain/             # BilletEntree, règles, événements
(BilletEmis)
|   |   └── infrastructure/     # JPA, QR, impression thermique
|   ├── evaluation/
|   ├── finance/
|   ├── communication/
|   └── ...
├── src/main/resources/db/migration/   # scripts Flyway V1__init.sql ...
└── src/test/java/                     # unitaires, intégration,
ModularityTests
```

## Frontend (Angular, espace de travail Nx)

```
frontend/
├── apps/
|   ├── backoffice/        # direction, secrétariat, profs, compta
|   ├── parent/            # PWA parent et élève
|   └── vie-scolaire/      # poste surveillant, écran billets
└── libs/
    ├── api-client/        # généré depuis OpenAPI
    ├── ui/                # composants partagés, thème par école
    ├── auth/              # OIDC, intercepteurs, gardes de route
    └── feature-*/         # un dossier par module métier
```

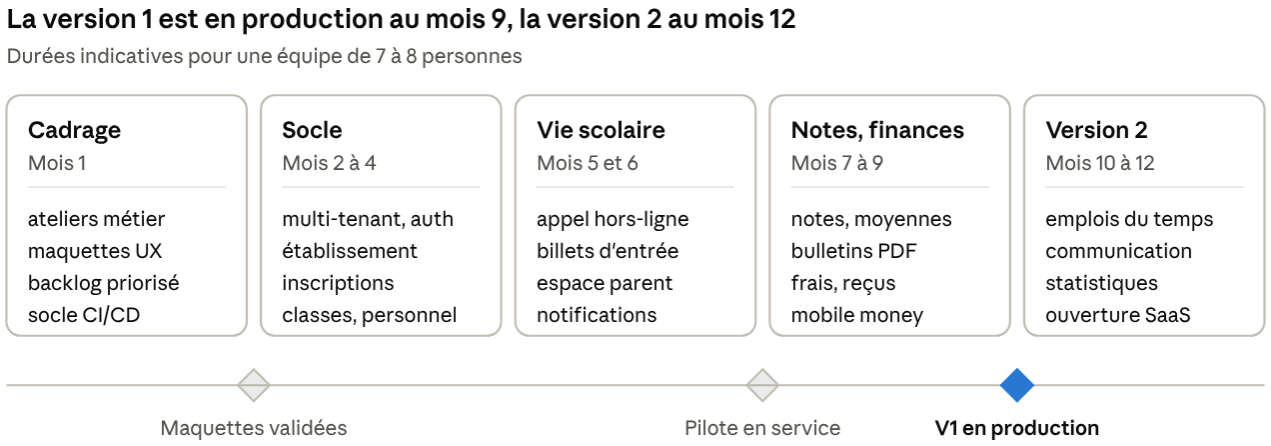
## Conventions d'équipe

- Langue : vocabulaire métier en français dans le code ( `BilletEntree` , `Inscription` ), termes techniques en anglais ( `Repository` , `Controller` ).
- DTO distincts des entités ; jamais d'entité JPA exposée par l'API.
- Couverture de tests : 80 % minimum sur `domain` et `application` ; tests d'intégration pour chaque endpoint.
- Test d'isolement multi-tenant obligatoire : un utilisateur de l'école A ne lit aucune donnée de l'école B.

- Branches courtes, revue de code obligatoire, commits conventionnels, versionnage sémantique.
- Décisions d'architecture traçées en ADR (un court fichier par décision).

## 9. Feuille de route et estimation

La version 1 complète (vie scolaire, billets, espace parent, notes, bulletins, finances) est en production environ 9 mois après le démarrage ; l'école pilote utilise dès le mois 6 l'appel, les billets et l'espace parent.




feuille de route · 5 phases, 3 jalons

Chaque phase se termine par une démonstration à l'école pilote ; on ne passe au jalon suivant qu'après validation par ses utilisateurs.

### Équipe recommandée

| Profil | Effectif | Rôle |
|---|---|---|
| Architecte / lead technique Java | 1 | Socle, sécurité, multi-tenant, revues de code |
| Développeurs backend Spring | 2 | Modules métier, intégrations |
| Développeurs Angular | 2 | Back-office, PWA parent, poste vie scolaire |
| Designer UX/UI | 0,5 | Maquettes, tests avec parents et enseignants |
| Testeur QA | 1 | Scénarios métier, tests automatisés de bout en bout |
| DevOps | 0,5 | Infrastructure, CI/CD, supervision, sauvegardes |
| Référent métier (côté école) | 0,5 | Priorités, règles de calcul, recette |

Ces durées sont indicatives : elles doublent environ avec une équipe de 3 personnes, et se précisent après la phase de cadrage.

## 10. Risques et décisions à valider

### Principaux risques

| Risque | Impact | Parade |
|---|---|---|
| Faible adoption par les enseignants | Données incomplètes, parents déçus | Appel en 3 clics sur téléphone, formation sur site, école pilote associée dès la conception |
| Connexion Internet instable | Saisies perdues, frustration | Mode hors-ligne pour l'appel et les billets, pages légères |
| Fuite de données entre écoles | Perte de confiance, sanction CDP | Double isolement Hibernate + RLS, tests automatiques, audit |
| Règles de calcul des moyennes différentes selon les écoles | Bulletins faux | Moteur de calcul paramétrable, validation sur bulletins réels de l'école pilote |
| Dépendance à un fournisseur de paiement ou SMS | Interruption de service | Interfaces abstraites, deux fournisseurs configurés |

| Risque | Impact | Parade |
|---|---|---|
| Périmètre trop large dès la V1 | Retard, budget dépassé | Strict respect des priorités P1/P2/P3 |

### Décisions à confirmer de votre côté

- [ ] École pilote : laquelle, et quel cycle (primaire, collège ou lycée) en premier ?
- [ ] Format exact des bulletins actuels et règles de calcul (moyennes, rangs, mentions) à reproduire.
- [ ] Billets : impression sur ticket thermique, affichage écran, ou les deux ?
- [ ] Opérateurs de paiement prioritaires (Wave, Orange Money, autre) et agrégateur.
- [ ] Hébergement : centre de données local ou cloud, et budget mensuel.
- [ ] Modèle commercial : abonnement par élève, par école, ou licence.
- [ ] Composition de l'équipe de développement disponible.
- [ ] Angular confirmé comme frontend (ou React si votre équipe le maîtrise mieux).

## Sources


- HeroDevs – versions et fins de support de Spring Boot
- eCorp IT – calendrier des versions Angular (v22)
- CIO Mag – loi sénégalaise sur les données personnelles